

MODULE 1 ASSIGNMENT

"How the Internet Delivers a Web Page"

Student Name: Deon J Manoj **Roll No:** 24 **Class:** CSA

Course: PCCST501 – Computer Networks **Module:** 1

Website chosen: vjcet.org (Viswajyothi College of Engineering and Technology)

1. Introduction

Loading a website feels instantaneous, but behind that single click a whole chain of networking machinery springs into action. Typing www.vjcet.org into a browser and hitting Enter sets off a fast, layered exchange between the user's device and a remote server: the domain name must first be turned into an address, a reliable channel has to be opened, the page itself has to be requested and delivered securely, and finally the browser has to make sense of what arrived and paint it on screen. This report walks through each of those stages using the official website of Viswajyothi College of Engineering and Technology (VJCET) as a concrete, real-world example.

2. The TCP/IP Protocol Stack

Figure 1: TCP/IP Four-Layer Protocol Stack

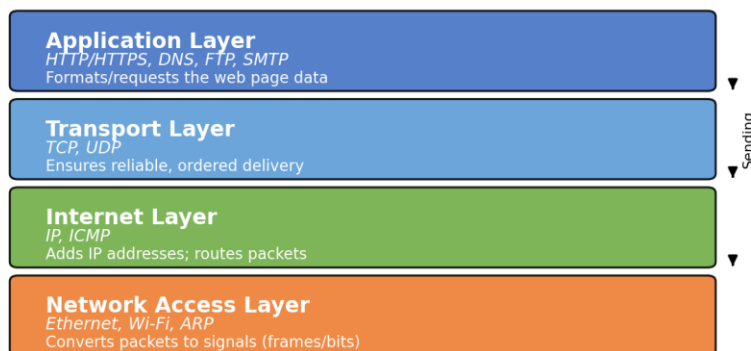


Figure 1: TCP/IP four-layer protocol stack

Layer	Protocols	Role in loading vjcet.org
Application	HTTP/HTTPS, DNS	Where the browser itself lives: it puts together the request for VJCET's homepage and later makes sense of the HTML/CSS/JS that comes back, while DNS quietly resolves vjcet.org to an address.
Transport	TCP, UDP	Breaks the request into numbered segments and takes responsibility for getting every one of them to VJCET's server, in order, opening the connection first via the three-way handshake.
Internet	IP, ICMP	Stamps each segment with source and destination IP addresses and lets routers along the way pass the resulting packet toward VJCET's hosting server.
Network Access	Ethernet, Wi-Fi, ARP	Handles the last physical step — turning packets into frames and pushing them out as Wi-Fi radio waves or Ethernet signals over the local link.

Encapsulation: on its way down the stack, the outgoing request picks up a new header at every layer — the plain HTTP message becomes a TCP segment, then an IP packet, then a link-layer frame — much like a letter being placed inside successively larger envelopes. VJCET's server simply undoes this in reverse (de-encapsulation), peeling off one header at a time until the original HTTP request is exposed. TCP, wrapped inside HTTPS, is the natural choice here rather than UDP: nobody wants half of an admissions notice or a corrupted syllabus table, so guaranteed, ordered delivery matters more than raw speed. UDP earns its keep elsewhere — live video calls or the DNS lookups described next — where a little data loss is tolerable but delay is not.

3. Role of DNS

Routers and switches move traffic using numeric addresses, not names, so something has to bridge the gap between a human-friendly domain like www.vjcet.org and the IP address the network actually understands. That bridge is the Domain Name System (DNS), effectively a distributed directory that the browser consults before it can send a single byte to VJCET's server.

Figure 2: DNS Resolution Process for vjcet.org

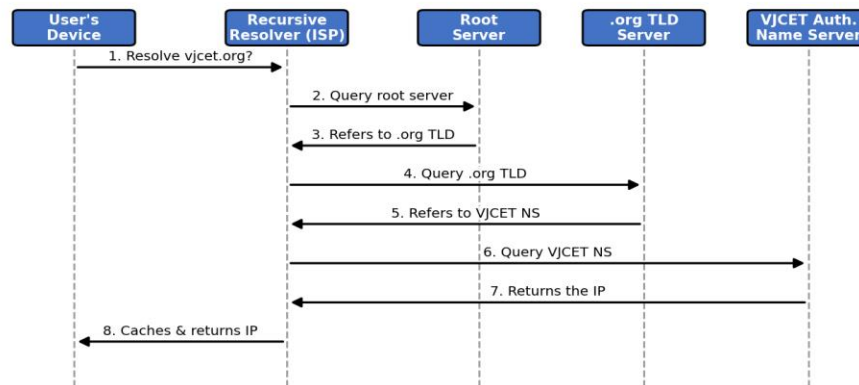


Figure 2: DNS resolution process for vjcet.org

Resolution begins locally — the browser and operating system check their own caches for a recent lookup. On a cache miss, the request is handed to a recursive resolver (run by the ISP, or a public one such as 8.8.8.8), which does the legwork of climbing the DNS hierarchy on the browser's behalf: it is pointed from a root server to the .org TLD server, and from there to VJCET's own authoritative name server, which finally hands back the real IP address (shown here as 203.0.113.42, a stand-in used purely for illustration). That answer is then cached for a duration set by its TTL, so the next visit to the site can skip straight past this step.

Record	Purpose
A / AAAA	Maps vjcet.org to an IPv4 / IPv6 address
CNAME	Aliases www.vjcet.org to the canonical domain vjcet.org
NS / TTL	NS names the authoritative servers; TTL sets how long resolvers cache the record

4. HTTP: Requesting and Receiving the Webpage

Once the IP address is in hand and the TCP handshake has opened a channel, the actual conversation can begin. The browser issues a GET request for the homepage; VJCET's server answers with a status line, a set of headers, and the HTML itself. From there the browser re-reads the markup, spots references to stylesheets, scripts, images, and fonts, and quietly fires off a fresh request for each one before finally assembling everything into the page the visitor sees.

Request

GET / HTTP/1.1 Host: www.vjcet.org User-Agent: Mozilla/5.0 Accept: text/html

Response

HTTP/1.1 200 OK Content-Type: text/html; charset=UTF-8 Set-Cookie: session_id=xyz789; Secure; HttpOnly

In reality this whole exchange happens over HTTPS rather than bare HTTP: a TLS handshake runs first, scrambling the traffic and confirming that the server on the other end is genuinely VJCET's — a safeguard that matters the moment a student types personal information into an admission-enquiry form or the login portal.

Code	Meaning	Example on vjcet.org
200	OK	Homepage loads normally
301/302	Redirect	vjcet.org → www.vjcet.org
304	Not modified	Cached college logo reused
404	Not found	Broken link to an old notice
500	Server error	Fault during high-traffic admissions

5. How TCP Provides Reliable Delivery

HTTP simply trusts that whatever it hands off will arrive intact — that trust is earned by TCP underneath it. Before a single byte moves, a three-way handshake (SYN, SYN-ACK, ACK) lines up both sides; every byte afterward carries a sequence number so the pieces can be put back in order even if they arrive scrambled; the receiver sends acknowledgements back so the sender knows what got through and can retransmit anything that timed out; a sliding window throttles the sender so it never floods a slower receiver; and checksums catch corrupted segments so they get discarded and resent rather than silently accepted.

6. Client–Server or Peer-to-Peer?

Every visitor to vjcet.org plays the same fixed role: the browser only ever asks, and a single, centrally managed web server only ever answers. No two visitors' browsers ever talk to each other to piece together the homepage, which rules out a peer-to-peer arrangement and firmly places the site in the Client–Server camp — in contrast to something like BitTorrent, where each participant can act as both requester and provider of data.

Aspect	Client–Server (vjcet.org)	Peer-to-Peer (BitTorrent)
Roles	Fixed: client requests, server responds	Interchangeable: every node is client & server
Control	Centralized at VJCET's server	Decentralized across peers
Failure	Single point of failure at the server	Survives loss of individual peers

7. HTTP vs FTP

Aspect	HTTP	FTP
Purpose	Transfers web pages/resources for browsing	Transfers files for storage/download
Connections	Single connection for requests and data	Separate control (port 21) & data (port 20)
State	Stateless by default (cookies add state)	Stateful for the session duration
Authentication	Often none for public content	Typically needs a username & password
Typical use	Loading vjcet.org	Uploading VJCET's site files to the host

8. How Cookies Improve User Experience

On its own, HTTP forgets a visitor the instant a response is sent — every request looks brand new to the server. Cookies patch over this by giving the server a way to leave a small note in the browser, which then gets handed back automatically on every later request to the same site. **Example:** a VJCET student who signs in to the student portal to view results gets a session cookie in return; that cookie is what keeps them recognised as they click across to notices or department pages, so they aren't asked to log in again at every stop.

9. End-to-End Communication Flow

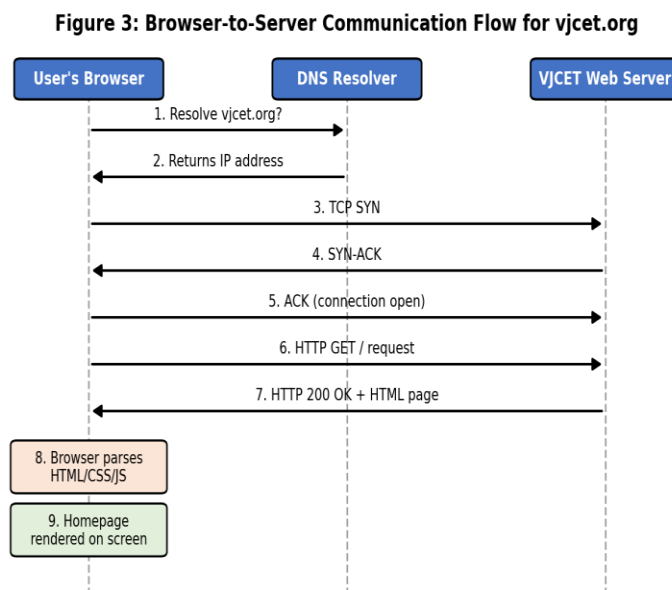


Figure 3: Complete browser-to-server communication flow for loading vjcet.org

10. Conclusion

What looks like one simple action — opening vjcet.org — is really the end product of several protocols quietly cooperating. DNS supplies the address behind the name, TCP makes sure nothing gets lost or scrambled along the way, IP handles routing across intervening networks, and HTTP gives browser and server a shared language for the request and response. Cookies then patch in the memory that HTTP otherwise lacks, enabling conveniences like a persistent student-portal login. None of this is visible to the person who typed the address, yet it is exactly this layered, dependable design that lets a college website load correctly and securely every time — which is why application-layer protocols deserve to be understood rather than taken for granted.

11. References

- Tanenbaum, A. S., & Wetherall, D. J. *Computer Networks* (5th Edition). Pearson Education
- Stallings, W. *Data and Computer Communications* (11th Edition). Pearson
- Mozilla Developer Network (MDN Web Docs) – HTTP documentation
- IETF RFC 793 – Transmission Control Protocol; RFC 1035 – Domain Names
- Viswajyothi College of Engineering and Technology – official website, vjcet.org